

Baterías

Este es un problema interactivo

Tras el primer día de examen, regresan todos cansados al Holiday Inn en el que se hospedan todas las delegaciones. Juan en particular está tan cansado que aunque sea de día, se acuesta y se queda dormido.

Juan despertó a mitad de la noche, varias horas después. Al tratar de prender las luces, se da cuenta de que todo el hotel se quedó sin energía eléctrica. Para colmo, Juan se quedó sin batería en el celular, está sumamente hambriento y necesita comer a la brevedad, pero dado a sus problemas de orientación en la oscuridad, no lo puede hacer si no hay luz.

Tú eres compañero de cuarto de Juan y realmente quieres ayudarlo. Para tu buena suerte, en la habitación hay una linterna y n baterías. Recuerdas que cuando les mostraron la habitación les dijeron que habían a baterías que sí funcionaban y b baterías que no. La linterna de la que disponen tiene capacidad para **DOS** baterías, y solo prenderá si ambas baterías ingresadas funcionan.

Tú quieres encontrar 2 baterías que funcionen, y para ello, puedes introducir 2 baterías en la linterna. Si ambas baterías funcionan, la linterna prenderá, y si no, no lo hará.

Como buen informático que eres, te preguntas cómo podrías probar las baterías tal que con la menor cantidad de intentos, la linterna prenda y se la puedas dar a Juan para que se pueda bañar.

¡Por favor, enciende la linterna con la mínima cantidad de intentos!

Detalles de Implementación

Deberás implementar la siguiente función:

```
void busca_baterias(int n, int a);
```

Esta función recibe dos parámetros:

- Un entero n , la cantidad de baterías.
- Un entero a , la cantidad de baterías válidas.

Y regresa 1 si la linterna se enciende, 0 si no es así.

En tu código, podrás invocar la siguiente función para hacer una pregunta:

```
int linterna(int i, int j);
```

La función recibe los siguientes parámetros:

- Un entero i , ingresas la batería i .
- Un entero j , ingresas la batería j .

Cuando llames esta función, los valores tienen que ser mayores o iguales a 1 y menores o iguales a N , y el valor de i debe ser distinto al de j . De lo contrario, la linterna no prenderá.

La función regresa un entero, un 1 si la linterna se enciende, y un 0 en caso contrario.

Para poder llevar a cabo la interacción, debes incluir la librería *baterias.h* en tu código, con el siguiente comando: `#include "baterias.h"`. A continuación hay un ejemplo de cómo se vería el código que envías (**revisa los archivos adjuntos, ahí también encontrarás un archivo que emula el evaluador para poder probar tu código**).

```
#include "baterias.h"
#include <bits/stdc++.h>
using namespace std;

int busca_baterias(int n, int a) {
    // programa tu solución aquí.
}
```

El evaluador es **adaptativo**, esto significa que los estados de las baterías no están fijos al inicio del proceso y pueden cambiar dependiendo de tus preguntas. Sin embargo, se garantiza que siempre existen a baterías que funcionan y que es consistente con todas las preguntas previas.

Ejemplo

- El evaluador corre tu programa, y llama la función: *busca_baterias(3, 2)*
- Digamos que las baterías 1 y 3 son las baterías que funcionan, el resto no. Estas son todas las posibles llamadas que puede hacer tu código en este caso:

Parámetros	Proceso	valor que regresa la función
<i>linterna(1, 2)</i>	Preguntas por la batería 1 y la batería 2	0
<i>linterna(1, 3)</i>	Preguntas por la batería 1 y la batería 3	1
<i>linterna(2, 1)</i>	Preguntas por la batería 2 y la batería 1	0
<i>linterna(2, 3)</i>	Preguntas por la batería 2 y la batería 3	0
<i>linterna(3, 1)</i>	Preguntas por la batería 3 y la batería 1	1
<i>linterna(3, 2)</i>	Preguntas por la batería 3 y la batería 2	0

Consideraciones

- $2 \leq n \leq 30$.
- $2 \leq a \leq n$
- $a + b = n$
- Las baterías están indexadas a partir del 1.

Subtareas

- Subtarea 1 (4 puntos): $a = 2$
- Subtarea 2 (6 puntos): $a = 3$
- Subtarea 3 (12 puntos): $a = b$
- Subtarea 4 (15 puntos): $a = b + 1$
- Subtarea 5 (18 puntos): $a > b$ y no conoces a , en este caso se te dará en el input como $a = -1$, y a lo más puedes usar $b + 2$ llamadas a la función *linterna()*
- Subtarea 6 (20 puntos): No conoces a . En este caso se te dará el input como $a = -1$. Puedes usar a lo más n^2/a llamadas a la función *linterna()*
- Subtarea 7 (25 puntos): Sin restricciones adicionales.

Evaluador de prueba

En el apartado de *Adjuntos* vendrá un documento llamado *grader.cpp*. Dicho documento ayudará a simular tu código.

Examen del día 2 de la 30a OMI

Acceso permitido a (SEBASREYES) **Sair** Automático (español)

no hay ningún enunciado disponible

Algunos detalles

Tipo	Communication	
Límite de tiempo	3,000 segundos	
Límite de memoria	256 MiB	
Comandos de compilación	C++11 / g++	/usr/bin/g++ -DEVAL -std=gnu++11 -O2 -pipe -static -s -o baterias stub.cpp baterias.cpp
	C++14 / g++	/usr/bin/g++ -DEVAL -std=gnu++14 -O2 -pipe -static -s -o baterias stub.cpp baterias.cpp
	C++17 / g++	/usr/bin/g++ -DEVAL -std=gnu++17 -O2 -pipe -static -s -o baterias stub.cpp baterias.cpp
	C++20 / g++	/usr/bin/g++ -DEVAL -std=gnu++20 -O2 -pipe -static -s -o baterias stub.cpp baterias.cpp
	C11 / gcc	/usr/bin/gcc -DEVAL -std=gnu11 -O2 -pipe -static -s -o baterias stub.c baterias.c -lm
	Pascal / fpc	/usr/bin/fpc -dEVAL -O2 -XSs -obaterias stub.pas

Adjuntos

grader.cpp 792 bytes
C++ source code

Dentro del documento, te encontrarás con código que contiene la función que deberás programar.

```
Start here X grader.cpp X
1 #include<bits/stdc++.h>
2
3 int linterna(int a, int b);
4
5
6
7
8
9
10 // TU CODIGO EMPIEZA AQUI
11 #include<bits/stdc++.h>
12 using namespace std;
13
14
15 void busca_baterias(int n, int a) {
16
17     return;
18 }
19
20 // TU CODIGO TERMINA AQUI
21
22
23
24
25
26
27 bool WA = true;
28 vector<int> v;
29 int cnt = 0;
30
```

No hace falta preocuparse por todo lo que esté fuera del rectángulo rojo. Abajo están las funciones que usa el programa para calificar tu código.

Cuando quieras probar tu código, corre el documento completo.

El documento leerá un entero N , seguido de N enteros $a_1, a_2, a_3 \dots, a_n$, los cuales son los indicadores para saber si la i -ésima batería funciona o no.

- 1 = Batería que sí funciona.
- 0 = Batería que no funciona.

```

1 #include<bits/stdc++.h>
2
3 int linterna(int a, int b);
4
5
6
7
8
9
10 // TU CODIGO EMPIEZA AQUI
11
12 #include<bits/stdc++.h>
13 using namespace std;
14
15
16 void busca_baterias(int n, int a) {
17
18     return;
19 }
20
21 // TU CODIGO TERMINA AQUI
22
23
24
25
26
27 bool WA = true;
28 vector<int> v;
29 int cnt = 0;
30
31 int linterna(int a, int b) {

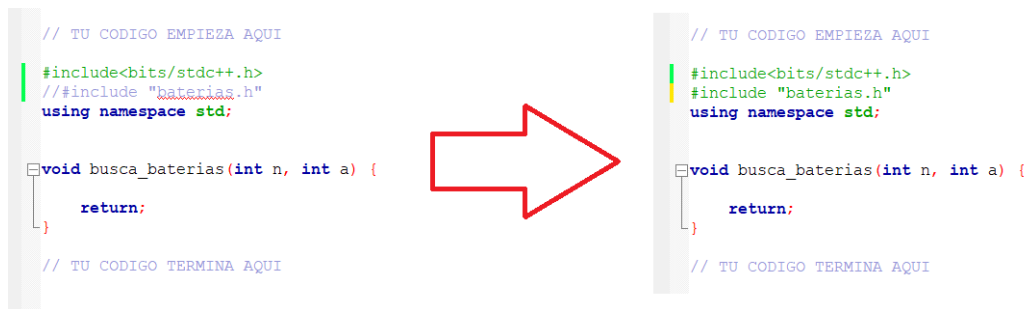
```

```

6
0 1 1 0 0 1
WA: No encontraste 2 baterias buenas :(
Process returned 0 (0x0) execution time : 6.242 s
Press any key to continue.

```

Si das una entrada válida, el documento te dirá si tu código generó una respuesta correcta o errónea, además del número de preguntas que le tomó a tu código obtener dicha respuesta.



Una vez que tengas tu código finalizado, deberás copiar lo que escribiste dentro de los “TU CODIGO VA AQUI” y pegarlo en un nuevo documento. Después de pegarlo, deberás borrar las “//” de “#include “baterias.h”” para tener tu código listo para enviar.